

Modül 24

Micro-Frontend

Module Federation & Architecture

Modül:	24 / 30
Ders Sayısı:	7 ders
Seviye:	İleri
Versiyon:	Angular v21

Hazırlayan: Claude • Türkçe Angular v21 Eğitimi

Modül 24'e Hoş Geldin

Bu modül senin için özellikle önemli — TSE'de Flexi.Kalite, APM, FATURA gibi remote modülleri olan micro-frontend mimari kullanıyorsun. Bu modülde Module Federation, shell app, shared state ve production patterns'leri derinlemesine işliyoruz.

Öğrenecekler:

- ✓ Micro-frontend nedir, ne zaman kullanılır
- ✓ Module Federation kavramı
- ✓ @angular-architects/native-federation setup
- ✓ Shell + Remote app pattern
- ✓ Shared dependencies management
- ✓ Cross-app state sharing
- ✓ Routing ve navigation patterns

Micro-Frontend Nedir?

Big app problemine modern çözüm.

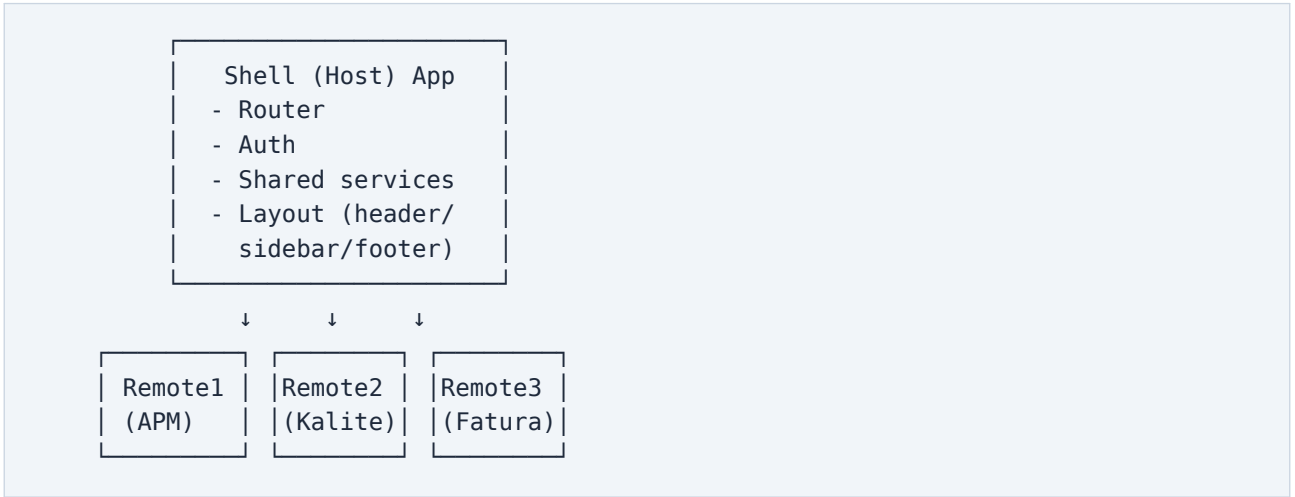
Hangi Problemi Çözüyor?

Senaryo: 50+ kişilik 5 takım, tek monolithic Angular app üzerinde çalışıyor. Build 20 dakika, deploy 1 saat, conflict'ler sürekli, takım koordinasyonu kabus.

Çözüm: Micro-Frontend

Yaklaşım	Monolith	Micro-Frontend
Build time	20 dk	2 dk per app
Deploy	Tüm app	Ayrı ayrı
Takım	Coordinated	Bağımsız
Tech stack	Tek	Esnek (Angular + React mümkün)
Risk	Tüm app down	Sadece bir parça

Mimari



TSE Senaryon

Shell App

└─ KALITE	(Quality Management – Flexi.Kalite)
└─ BELGE	(Document Management – Flexi.Belge)
└─ APM	(Application Performance)
└─ TSE_GOREV	(Task Management)
└─ TIP_ONAY	(Approval Workflow)
└─ FATURA	(Invoice Management)

Her remote bağımsız:

- Ayrı repo

- Ayrı takım

- Ayrı deploy cycle

- Ayrı versiyon

Module Federation Kavramı

Webpack 5'in micro-frontend için temel feature'ı.

❑ Module Federation Nedir?

Module Federation: Runtime'da bir uygulamanın diğerinin modüllerini yüklemesine olanak veren Webpack 5 feature'ı. Build time'da değil, kullanıcı görüntülediğinde.

❑ Native Federation (Angular Önerilen)

```
# Native Federation = Browser-native dynamic import + import maps  
# Webpack dependency yok, daha esnek, daha hızlı  
  
npm install -D @angular-architects/native-federation
```

❑ İki Tür App

Tür	Rol	Örnek
Host (Shell)	Remote'ları yükler	Ana app
Remote	Modülleri sunar	KALITE, APM
Mixed	İkisi de	Aynı anda host + remote

Native Federation Setup

Shell ve remote'u yapılandırma.

⚙️ Shell App Setup

```
# Shell projesinde
ng add @angular-architects/native-federation --project shell --type host

# Otomatik oluşur:
# - federation.config.js
# - bootstrap.ts
# - main.ts güncellenir
```

📄 Shell — federation.config.js

```
module.exports = require('@angular-architects/native-federation/config')
  .withNativeFederation({
    name: 'shell',

    // Shared dependencies (vendor sharing)
    shared: {
      '@angular/core': { singleton: true, strictVersion: true },
      '@angular/common': { singleton: true, strictVersion: true },
      '@angular/router': { singleton: true, strictVersion: true },
      'rxjs': { singleton: true, strictVersion: true },
    },
  });
```

📄 Remote App Setup

```
ng add @angular-architects/native-federation --project kalite --type remote
```

📄 Remote — federation.config.js

```
module.exports = require('@angular-architects/native-federation/config')
  .withNativeFederation({
    name: 'kalite',

    // Export edilecek modüller
    exposes: {
      './Module': './src/app/kalite/kalite.module.ts',
      './Routes': './src/app/kalite/kalite.routes.ts',
      './KaliteComponent': './src/app/kalite/kalite.component.ts',
    },

    shared: {
      '@angular/core': { singleton: true, strictVersion: true },
      '@angular/common': { singleton: true, strictVersion: true },
      '@angular/router': { singleton: true, strictVersion: true },
      'rxjs': { singleton: true, strictVersion: true },
    },
  });
```

Shell Routing — Remote Loading

Shell'in remote'ları yüklemesi.

❏ **federation.manifest.json**

```
{
  "kalite": "http://localhost:4201/remoteEntry.json",
  "apm": "http://localhost:4202/remoteEntry.json",
  "fatura": "http://localhost:4203/remoteEntry.json"
}

# Production:
{
  "kalite": "https://kalite.tse.org.tr/remoteEntry.json",
  "apm": "https://apm.tse.org.tr/remoteEntry.json"
}
```

❏ **Shell Routes**

```
// shell/app.routes.ts
import { Routes } from '@angular/router';
import { loadRemoteModule } from '@angular-architects/native-federation';

export const routes: Routes = [
  { path: '', component: HomeComponent },

  {
    path: 'kalite',
    loadChildren: () =>
      loadRemoteModule({
        remoteName: 'kalite',
        exposedModule: './Routes'
      }).then(m => m.KALITE_ROUTES)
  },

  {
    path: 'apm',
    loadChildren: () =>
      loadRemoteModule({
        remoteName: 'apm',
        exposedModule: './Routes'
      }).then(m => m.APM_ROUTES)
  },

  // ... diğer remote'lar
];
```

□ Bootstrap

```
// shell/main.ts
import { initFederation } from '@angular-architects/native-federation';

initFederation('/assets/federation.manifest.json')
  .catch(err => console.error(err))
  .then(_ => import('./bootstrap'))
  .catch(err => console.error(err));
```

Shared State & Services

Remote'lar shell'in service'lerini nasıl kullanır?

□ Shared Services

```
// Shell'de tanımlanan service
@Injectables({ providedIn: 'root' })
export class AuthService {
  user = signal<User | null>(null);
  // ...
}

// Remote'tan kullanmak için iki yol:

// YOL 1: Singleton paylaş (federation.config.js)
shared: {
  '@my-org/shared-lib': {
    singleton: true,
    strictVersion: true,
    requiredVersion: '1.0.0'
  }
}

// YOL 2: Custom Element ile shell expose
// Karmaşık, genelde monorepo + shared lib daha iyi
```

□ Shared Library Pattern (Önerilen)

```
// monorepo yapısı
my-app/
  shell/
  remotes/
    kalite/
    apm/
    fatura/
  libs/
    shared-models/      ← User, Role, etc.
    shared-services/    ← AuthService, ConfigService
    shared-ui/          ← Button, Card, Modal components

// libs/shared-services/src/lib/auth.service.ts
@Injectable({ providedIn: 'root' })
export class AuthService { /* ... */ }

// Hem shell hem remote'lar bu lib'i import eder
import { AuthService } from '@my-app/shared-services';
```

□ Cross-App Communication

```
// Shared event bus
@Injectable({ providedIn: 'root' })
export class AppEventBus {
  private events = new Subject<AppEvent>();

  events$ = this.events.asObservable();

  emit(type: string, payload?: any) {
    this.events.next({ type, payload, source: 'unknown' });
  }
}

// Shell'de
this.eventBus.emit('user-logged-out');

// Remote'ta listen
this.eventBus.events$
  .pipe(filter(e => e.type === 'user-logged-out'))
  .subscribe(() => this.cleanup());
```

Best Practices

Production-grade micro-frontend mimarisi.

☐ Do's

- ✓ Monorepo kullan (Nx, Turborepo) — shared lib management kolay
- ✓ Shared lib ile common code paylaş
- ✓ Versioning stratejisi belirle (semantic versioning)
- ✓ Contract testing — remote'ların interface'i sabit
- ✓ Independent deployment — her remote ayrı pipeline
- ✓ Design system tek yerden (shared-ui)
- ✓ Error boundary — bir remote down olursa diğerleri çalışsın
- ✓ Lazy load remote'lar — sadece kullanıldıkça yüklen

☐ Don'ts

- ✗ Tight coupling — remote'lar birbirine direkt bağımlı olmasın
- ✗ State sync hell — global state çoğunlukla shell'de
- ✗ Version skew — Angular versiyonu farklı remote'larda
- ✗ Style conflicts — CSS isolation gerek (web component, shadow DOM)
- ✗ Too many remotes — 3-5 ideal, 20+ overhead

☐ Error Boundary Pattern

```
@Component({
  template: `
    @if (loadError()) {
      <div class="error-fallback">
        Modül yüklenemedi. <button (click)="retry()">Tekrar Dene</button>
      </div>
    } @else {
      <router-outlet />
    }
  `,
})
export class RemoteWrapperComponent {
  loadError = signal(false);

  constructor() {
    // Loading errors
  }

  retry() { window.location.reload(); }
}
```

TSE Pattern — Senin Mimarinde

Senin gerçek dünya kullanım senaryon.

□ TSE Mimarisi

```
Shell (Ana Portal)
├── routes:
│   ├── /apm          → APM Remote
│   ├── /kalite       → Flexi.Kalite Remote
│   ├── /belge        → Flexi.Belge Remote (BFF)
│   ├── /gorev        → TSE_GOREV Remote
│   ├── /onay         → TIP_ONAY Remote
│   └── /fatura        → FATURA Remote
└── shared:
    ├── @tse/shared-models (User, Role, Permission)
    ├── @tse/shared-services (AuthService, ConfigService, BelgeApi)
    ├── @tse/shared-ui (PrimeNG wrapper components)
    └── @tse/shared-utils (formatters, validators)
```

□ Önerim — Senin İçin

- ✓ Nx monorepo ile başla — TSE'de varsa harika
- ✓ Shared services mutlaka library olarak çıkar
- ✓ PrimeNG theming tek yerde — tüm remote'larda tutarlı
- ✓ Auth & permissions shell'de — tek doğruluk kaynağı
- ✓ BelgeApi service shared lib'de — tüm modüller kullanır
- ✓ Independent deploy — her modül ayrı release cycle'da
- ✓ Smoke tests her deploy sonrası — shell + remote integration

⚙ PrimeNG Sharing

```
// federation.config.js (her remote ve shell'de)
shared: {
  // Angular core
  '@angular/core': { singleton: true, strictVersion: true },
  '@angular/common': { singleton: true, strictVersion: true },
  '@angular/router': { singleton: true, strictVersion: true },
  '@angular/forms': { singleton: true, strictVersion: true },

  // PrimeNG – singleton ÖNEMLİ
  'primeng': { singleton: true, strictVersion: true },
  'primeicons': { singleton: true },

  // RxJS
  'rxjs': { singleton: true, strictVersion: true },

  // TSE shared libs
  '@tse/shared-models': { singleton: true },
  '@tse/shared-services': { singleton: true },
  '@tse/shared-ui': { singleton: true },
}
```

Modül 24 Özeti

Senin günlük çalıştığın micro-frontend mimari artık çok daha net. Module Federation'ın mantığı, shared state, error handling, ve production patterns — TSE bağlamında uygulayabilirsin.

Neler Öğrendin?

- ✓ Micro-frontend nedir, ne zaman kullanılır
- ✓ Module Federation kavramı
- ✓ Native Federation setup (shell + remote)
- ✓ Shell routing ve remote loading
- ✓ Shared state ve services
- ✓ Best practices ve anti-patterns
- ✓ TSE özelinde uygulama

Sıradaki Modül

Modül 25 — PrimeNG Derinlemesine. TSE'de kullandığın UI library — komponentler, theming, custom styling.